

Chapter 23

Fast Matrix Multiplication

A simple but extremely valuable bit of equipment in matrix multiplication consists of two plain cards, with a re-entrant right angle cut out of one or both of them if symmetric matrices are to be multiplied. In getting the element of the i th row and j th column of the product, the i th row of the first factor and the j th column of the second should be marked by a card beside, above, or below it.

— HAROLD HOTELLING, *Some New Methods in Matrix Calculation* (1943)

It was found that multiplication of matrices using punched card storage could be a highly efficient process on the Pilot ACE, due to the relative speeds of the Hollerith card reader used for input (one number per 16 ms.) and the automatic multiplier (2 ms.). While a few rows of one matrix were held in the machine the matrix to be multiplied by it was passed through the card reader. The actual computing and selection of numbers from store occupied most of the time between the passage of successive rows of the cards through the reader, so that the overall time was but little longer than it would have been if the machine had been able to accommodate both matrices.

— MICHAEL WOODGER, *The History and Present Use of Digital Computers at the National Physical Laboratory* (1958)

23.1. Methods

A fast matrix multiplication method forms the product of two $n \times n$ matrices in $O(n^\omega)$ arithmetic operations, where $\omega < 3$. Such a method is more efficient asymptotically than direct use of the definition

$$(AB)_{ij} = \sum_{k=1}^n a_{ik}b_{kj}, \quad (23.1)$$

which requires $O(n^3)$ operations. For over a century after the development of matrix algebra in the 1850s by Cayley, Sylvester, and others, this definition provided the only known method for multiplying matrices. In 1967, however, to the surprise of many, Winograd found a way to exchange half the multiplications for additions in the basic formula [1249, 1968]. The method rests on the identity, for vectors of even dimension n ,

$$x^T y = \sum_{i=1}^{n/2} (x_{2i-1} + y_{2i})(x_{2i} + y_{2i-1}) - \sum_{i=1}^{n/2} x_{2i-1}x_{2i} - \sum_{i=1}^{n/2} y_{2i-1}y_{2i}. \quad (23.2)$$

When this identity is applied to a matrix product AB , with x a row of A and y a column of B , the second and third summations are found to be common to the other inner products involving that row or column, so they can be computed once and reused. Winograd's paper generated immediate practical interest because on the computers of the 1960s floating point multiplication was typically two or three times slower than floating point addition. (On today's machines these two operations are usually similar in cost.)

Shortly after Winograd's discovery, Strassen astounded the computer science community by finding an $O(n^{\log_2 7})$ operations method for matrix multiplication ($\log_2 7 \approx 2.807$). A variant of this technique can be used to compute A^{-1} (see Problem 23.8) and thereby to solve $Ax = b$, both in $O(n^{\log_2 7})$ operations. Hence the title of Strassen's 1969 paper [1093, 1969], which refers to the question of whether Gaussian elimination is asymptotically optimal for solving linear systems.

Strassen's method is based on a circuitous way to form the product of a pair of 2×2 matrices in 7 multiplications and 18 additions, instead of the usual 8 multiplications and 4 additions. As a means of multiplying 2×2 matrices the formulae have nothing to recommend them, but they are valid more generally for block 2×2 matrices. Let A and B be matrices of dimensions $m \times n$ and $n \times p$ respectively, where all the dimensions are even, and partition each of A , B , and $C = AB$ into four equally sized blocks:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}. \quad (23.3)$$

Strassen's formulae are

$$\begin{aligned}
 P_1 &= (A_{11} + A_{22})(B_{11} + B_{22}), \\
 P_2 &= (A_{21} + A_{22})B_{11}, \\
 P_3 &= A_{11}(B_{12} - B_{22}), \\
 P_4 &= A_{22}(B_{21} - B_{11}), \\
 P_5 &= (A_{11} + A_{12})B_{22}, \\
 P_6 &= (A_{21} - A_{11})(B_{11} + B_{12}), \\
 P_7 &= (A_{12} - A_{22})(B_{21} + B_{22}), \\
 C_{11} &= P_1 + P_4 - P_5 + P_7, \\
 C_{12} &= P_3 + P_5, \\
 C_{21} &= P_2 + P_4, \\
 C_{22} &= P_1 + P_3 - P_2 + P_6.
 \end{aligned} \tag{23.4}$$

Counting the additions (A) and multiplications (M) we find that while conventional multiplication requires

$$mnpM + m(n-1)pA,$$

Strassen's algorithm, using conventional multiplication at the block level, requires

$$\frac{7}{8}mnpM + \left(\frac{7}{8}m(n-2)p + \frac{5}{4}mn + \frac{5}{4}np + \frac{8}{4}mp \right)A.$$

Thus, if m , n , and p are large, Strassen's algorithm reduces the arithmetic by a factor of about 7/8. The same idea can be used recursively on the multiplications associated with the P_i . In practice, recursion is only performed down to the "crossover" level at which any savings in floating point operations are outweighed by the overheads of a computer implementation.

To state a complete operation count, we suppose that $m = n = p = 2^k$ and that recursion is terminated when the matrices are of dimension $n_0 = 2^r$, at which point conventional multiplication is used. The number of multiplications and additions can be shown to be

$$M(k) = 7^{k-r}8^r, \quad A(k) = 4^r(2^r + 5)7^{k-r} - 6 \cdot 4^k. \tag{23.5}$$

The sum $M(k) + A(k)$ is minimized over all integers r by $r = 3$; interestingly, this value is independent of k . The total operation count for the "optimal" $n_0 = 8$ is less than

$$4 \cdot 7^k = 4 \cdot 2^{\log_2 7^k} = 4(2^k)^{\log_2 7} = 4n^{\log_2 7} = 4n^{2.807\ldots}.$$

Hence, in addition to having a lower exponent, Strassen's method has a reasonable constant.

Winograd found a variant of Strassen's formulae that requires the same number of multiplications but only 15 additions (instead of 18). This variant therefore has slightly smaller constants in the operation count for $n \times n$ matrices. For the product

(23.3) the formulae are

$$\begin{aligned}
 S_1 &= A_{21} + A_{22}, & M_1 &= S_2 S_6, & T_1 &= M_1 + M_2, \\
 S_2 &= S_1 - A_{11}, & M_2 &= A_{11} B_{11}, & T_2 &= T_1 + M_4, \\
 S_3 &= A_{11} - A_{21}, & M_3 &= A_{12} B_{21}, \\
 S_4 &= A_{12} - S_2, & M_4 &= S_3 S_7, \\
 S_5 &= B_{12} - B_{11}, & M_5 &= S_1 S_5, & C_{11} &= M_2 + M_3, \\
 S_6 &= B_{22} - S_5, & M_6 &= S_4 B_{22}, & C_{12} &= T_1 + M_5 + M_6, \\
 S_7 &= B_{22} - B_{12}, & M_7 &= A_{22} S_8, & C_{21} &= T_2 - M_7, \\
 S_8 &= S_6 - B_{21}, & & & C_{22} &= T_2 + M_5.
 \end{aligned} \tag{23.6}$$

Until the late 1980s there was a widespread view that Strassen's method was of theoretical interest only, because of its supposed large overheads for dimensions of practical interest (see, for example, [1028, 1988] and [953, 1992]). However, in 1970 Brent implemented Strassen's algorithm in Algol-W on an IBM 360/67 and concluded that in this environment, and with just one level of recursion, the method runs faster than the conventional method for $n \geq 110$ [158, 1970]. In 1988, Bailey compared his Fortran implementation of Strassen's algorithm for the Cray-2 with the Cray library routine for matrix multiplication and observed speedup factors ranging from 1.45 for $n = 128$ to 2.01 for $n = 2048$ (although 35% of these speedups were due to Cray-specific techniques) [52, 1988]. These empirical results, together with more recent experience of various researchers, show that Strassen's algorithm *is* of practical interest, even for n in the hundreds. Indeed, Fortran codes for (Winograd's variant of) Strassen's method have been supplied with IBM's ESSL library [653, 1988] and Cray's UNICOS library [662, 1989] since the late 1980s.

Strassen's paper raised the question, "What is the minimum exponent ω such that multiplication of $n \times n$ matrices can be done in $O(n^\omega)$ operations?" Clearly, $\omega \geq 2$, since each element of each matrix must partake in at least one operation. It was 10 years before the exponent was reduced below Strassen's $\log_2 7$. A flurry of publications, beginning in 1978 with Pan and his exponent 2.795 [915, 1978], resulted in reduction of the exponent to the current record 2.376, obtained by Coppersmith and Winograd in 1987 [272, 1987]. Figure 23.1 plots exponent versus time of publication (not all publications are represented in the graph); in principle, the graph should extend back to 1850!

Some of the fast multiplication methods are based on a generalization of Strassen's idea to bilinear forms. Let $A, B \in \mathbb{R}^{h \times h}$. A bilinear noncommutative algorithm over $\mathbb{R}$ for multiplying A and B with t "nonscalar multiplications" forms the product $C = AB$ according to

$$c_{ij} = \sum_{k=1}^t w_{h(i-1)+j,k} P_k, \tag{23.7a}$$

$$P_k = \left(\sum_{i,j=1}^h u_{ij}^{(k)} a_{ij} \right) \left(\sum_{i,j=1}^h v_{ij}^{(k)} b_{ij} \right), \quad k = 1:t, \tag{23.7b}$$

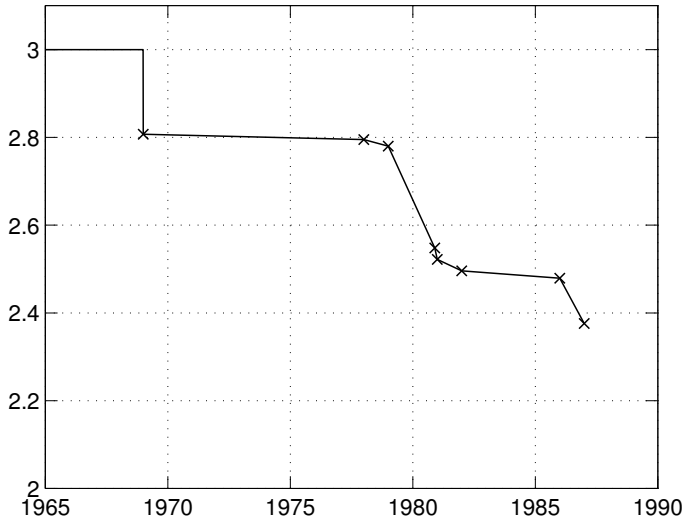


Figure 23.1. *Exponent versus time for matrix multiplication.*

where the elements of the matrices W , $U^{(k)}$, and $V^{(k)}$ are constants. This algorithm can be used to multiply $n \times n$ matrices A and B , where $n = h^k$, as follows: partition A , B , and C into h^2 blocks A_{ij} , B_{ij} , and C_{ij} of size h^{k-1} , then compute $C = AB$ by the bilinear algorithm, with the scalars a_{ij} , b_{ij} , and c_{ij} replaced by the corresponding matrix blocks. (The algorithm is applicable to matrices since, by assumption, the underlying formulae do not depend on commutativity.) To form the t products P_k of $(n/h) \times (n/h)$ matrices, partition them into h^2 blocks of dimension n/h^2 and apply the algorithm recursively. The total number of scalar multiplications required for the multiplication is $t^k = n^\alpha$, where $\alpha = \log_h t$.

Strassen’s method has $h = 2$ and $t = 7$. For 3×3 multiplication ($h = 3$), the smallest t obtained so far is 23 [760, 1976]; since $\log_3 23 \approx 2.854 > \log_2 7$, this does not yield any improvement over Strassen’s method. The method described in Pan’s paper [915, 1978] has $h = 70$ and $t = 143,640$, which yields $\alpha = \log_{70} 143,640 = 2.795 \dots$

In the methods that achieve exponents lower than 2.775, various intricate techniques are used. Laderman, Pan, and Sha [761, 1992] explain that for these methods “very large overhead constants are hidden in the ‘ O ’ notation”, and that the methods “improve on Strassen’s (and even the classical) algorithm only for immense numbers N ”.

A further method that is appropriate to discuss in the context of fast multiplication methods, even though it does not reduce the exponent, is a method for efficient multiplication of complex matrices. The clever formula

$$(a + ib)(c + id) = ac - bd + i[(a + b)(c + d) - ac - bd] \tag{23.8}$$

computes the product of two complex numbers using three real multiplications instead of the usual four. Since the formula does not rely on commutativity it extends to matrices. Let $A = A_1 + iA_2$ and $B = B_1 + iB_2$, where $A_j, B_j \in \mathbb{R}^{n \times n}$,

and define $C = C_1 + iC_2 = AB$. Then C can be formed using three real matrix multiplications as

$$\begin{aligned} T_1 &= A_1 B_1, & T_2 &= A_2 B_2, \\ C_1 &= T_1 - T_2, \\ C_2 &= (A_1 + A_2)(B_1 + B_2) - T_1 - T_2, \end{aligned} \tag{23.9}$$

which we will refer to as the “3M method”. This computation involves $3n^3$ scalar multiplications and $3n^3 + 2n^2$ scalar additions. Straightforward evaluation of the conventional formula $C = A_1 B_1 - A_2 B_2 + i(A_1 B_2 + A_2 B_1)$ requires $4n^3$ multiplications and $4n^3 - 2n^2$ additions. Thus, the 3M method requires strictly fewer arithmetic operations than the conventional means of multiplying complex matrices for $n \geq 3$, and it achieves a saving of about 25% for $n \geq 30$ (say). Similar savings occur in the important special case where A or B is triangular. This kind of clear-cut computational saving is rare in matrix computations!

IBM’s ESSL library and Cray’s UNICOS library both contain routines for complex matrix multiplication that apply the 3M method and use Strassen’s method to evaluate the resulting three real matrix products.

23.2. Error Analysis

To be of practical use, a fast matrix multiplication method needs to be faster than conventional multiplication for reasonable dimensions without sacrificing numerical stability. The stability properties of a fast matrix multiplication method are much harder to predict than its practical efficiency and need careful investigation.

The forward error bound (3.13) for conventional computation of $C = AB$, where $A, B \in \mathbb{R}^{n \times n}$, can be written

$$|C - \widehat{C}| \leq nu|A||B| + O(u^2). \tag{23.10}$$

Miller [850, 1975] shows that any polynomial algorithm for multiplying $n \times n$ matrices that satisfies a bound of the form (23.10) (perhaps with a different constant) must perform at least n^3 multiplications. (A polynomial algorithm is essentially one that uses only scalar addition, subtraction, and multiplication.) Hence Strassen’s method, and all other polynomial algorithms with an exponent less than 3, cannot satisfy (23.10). Miller also states, without proof, that any polynomial algorithm in which the multiplications are all of the form $(\sum_{i,j} u_{ij}^{(k)} a_{ij})(\sum_{i,j} v_{ij}^{(k)} b_{ij})$ must satisfy a bound of the form

$$|C - \widehat{C}| \leq f_n u \|A\| \|B\| + O(u^2). \tag{23.11}$$

It follows that any algorithm based on recursive application of a bilinear noncommutative algorithm satisfies (23.11); however, the all-important constant f_n is not specified. These general results are useful because they show us what types of results we can and cannot prove and thereby help to focus our efforts.

In the subsections below we analyze particular methods.

Throughout the rest of this chapter an unsubscripted matrix norm denotes

$$\|A\| := \max_{i,j} |a_{ij}|.$$

As noted in §6.2, this is not a consistent matrix norm, but we do have the bound $\|AB\| \leq n\|A\| \|B\|$ for $n \times n$ matrices.

23.2.1. Winograd's Method

Winograd's method does not satisfy the conditions required for the bound (23.11), since it involves multiplications with operands of the form $a_{ij} + b_{rs}$. However, it is straightforward to derive an error bound.

Theorem 23.1 (Brent). *Let $x, y \in \mathbb{R}^{n \times n}$, where n is even. The inner product computed by Winograd's method satisfies*

$$|x^T y - fl(x^T y)| \leq n\gamma_{n/2+4}(\|x\|_\infty + \|y\|_\infty)^2. \quad (23.12)$$

Proof. A straightforward adaptation of the inner product error analysis in §3.1 produces the following analogue of (3.3):

$$\begin{aligned} fl(x^T y) &= \sum_{i=1}^{n/2} (x_{2i-1} + y_{2i})(x_{2i} + y_{2i-1})(1 + \epsilon_i) \\ &\quad - \sum_{i=1}^{n/2} x_{2i-1}x_{2i}(1 + \alpha_i) - \sum_{i=1}^{n/2} y_{2i-1}y_{2i}(1 + \beta_i), \end{aligned}$$

where the ϵ_i , α_i , and β_i are all bounded in modulus by $\gamma_{n/2+4}$. Hence

$$\begin{aligned} |x^T y - fl(x^T y)| &\leq \gamma_{n/2+4} \sum_{i=1}^{n/2} (|x_{2i-1} + y_{2i}| |x_{2i} + y_{2i-1}| \\ &\quad + |x_{2i-1}x_{2i}| + |y_{2i-1}y_{2i}|) \\ &\leq \gamma_{n/2+4}(n/2)((\|x\|_\infty + \|y\|_\infty)^2 + \|x\|_\infty^2 + \|y\|_\infty^2) \\ &\leq n\gamma_{n/2+4}(\|x\|_\infty + \|y\|_\infty)^2. \quad \square \end{aligned}$$

The analogue of (23.12) for matrix multiplication is $\|AB - fl(AB)\| \leq n\gamma_{n/2+4} \times (\|A\| + \|B\|)^2$.

Conventional evaluation of $x^T y$ yields the bound (see (3.5))

$$|x^T y - fl(x^T y)| \leq \gamma_n |x|^T |y| \leq n\gamma_n \|x\|_\infty \|y\|_\infty. \quad (23.13)$$

The bound (23.12) for Winograd's method exceeds the bound (23.13) by a factor approximately $\|x\|_\infty / \|y\|_\infty + \|y\|_\infty / \|x\|_\infty$. Therefore Winograd's method is stable if $\|x\|_\infty$ and $\|y\|_\infty$ have similar magnitude, but potentially unstable if they differ widely in magnitude. The underlying reason for the instability is that Winograd's method relies on cancellation of terms $x_{2i-1}x_{2i}$ and $y_{2i-1}y_{2i}$ that can be much larger ($\|x\|_\infty^2$ and $\|y\|_\infty^2$) than the final answer ($|x^T y| \leq \|x\|_\infty \|y\|_1$); therefore the intermediate rounding errors can swamp the desired inner product.

A simple way to avoid the instability is to scale $x \leftarrow \mu x$ and $y \leftarrow \mu^{-1} y$ before applying Winograd's method, where μ , which in practice might be a power of the

machine base to avoid roundoff, is chosen so that $\|x\|_\infty \approx \|y\|_\infty$. When using Winograd's method for a matrix multiplication AB it suffices to carry out a single scaling $A \leftarrow \mu A$ and $B \leftarrow \mu^{-1}B$ such that $\|A\| \approx \|B\|$. If A and B are scaled so that $\tau^{-1} \leq \|A\|/\|B\| \leq \tau$ then

$$\|AB - fl(AB)\| \leq 2(\tau + 1)n\gamma_{n/2+4}\|A\|\|B\|.$$

23.2.2. Strassen's Method

Until recently there was a widespread belief that Strassen's method is numerically unstable. The next theorem, originally proved by Brent [158, 1970], shows that this belief is unfounded.

Theorem 23.2 (Brent). *Let $A, B \in \mathbb{R}^{n \times n}$, where $n = 2^k$. Suppose that $C = AB$ is computed by Strassen's method and that $n_0 = 2^r$ is the threshold at which conventional multiplication is used. The computed product $\hat{C}$ satisfies*

$$\|C - \hat{C}\| \leq \left[\left(\frac{n}{n_0} \right)^{\log_2 12} (n_0^2 + 5n_0) - 5n \right] u\|A\|\|B\| + O(u^2). \quad (23.14)$$

Proof. We will use without comment the norm inequality $\|AB\| \leq n\|A\|\|B\| = 2^k\|A\|\|B\|$.

Assume that the computed product $\hat{C} \approx AB$ from Strassen's method satisfies

$$\hat{C} = AB + E, \quad \|E\| \leq c_k u\|A\|\|B\| + O(u^2), \quad (23.15)$$

where c_k is a constant. In view of (23.10), (23.15) certainly holds for $n = n_0$, with $c_r = n_0^2$. Our aim is to verify (23.15) inductively and at the same time to derive a recurrence for the unknown constant c_k .

Consider C_{11} in (23.4), and, in particular, its subterm P_1 . Accounting for the errors in matrix addition and invoking (23.15), we obtain

$$\hat{P}_1 = (A_{11} + A_{22} + \Delta_A)(B_{11} + B_{22} + \Delta_B) + E_1,$$

where

$$\begin{aligned} |\Delta_A| &\leq u|A_{11} + A_{22}|, \\ |\Delta_B| &\leq u|B_{11} + B_{22}|, \\ \|E_1\| &\leq c_{k-1}u\|A_{11} + A_{22} + \Delta_A\|\|B_{11} + B_{22} + \Delta_B\| + O(u^2) \\ &\leq 4c_{k-1}u\|A\|\|B\| + O(u^2). \end{aligned}$$

Hence

$$\begin{aligned} \hat{P}_1 &= P_1 + F_1, \\ \|F_1\| &\leq (8 \cdot 2^{k-1} + 4c_{k-1})u\|A\|\|B\| + O(u^2). \end{aligned}$$

Similarly,

$$\hat{P}_4 = A_{22}(B_{21} - B_{11} + \Delta_B) + E_4,$$

where

$$\begin{aligned} |\Delta_B| &\leq u|B_{21} - B_{11}|, \\ \|E_4\| &\leq c_{k-1}u\|A_{22}\| \|B_{21} - B_{11} + \Delta_B\| + O(u^2), \end{aligned}$$

which gives

$$\begin{aligned} \hat{P}_4 &= P_4 + F_4, \\ \|F_4\| &\leq (2 \cdot 2^{k-1} + 2c_{k-1})u\|A\| \|B\| + O(u^2). \end{aligned}$$

Now

$$\hat{C}_{11} = fl(\hat{P}_1 + \hat{P}_4 - \hat{P}_5 + \hat{P}_7),$$

where $\hat{P}_5 =: P_5 + F_5$ and $\hat{P}_7 =: P_7 + F_7$ satisfy exactly the same error bounds as $\hat{P}_4$ and $\hat{P}_1$, respectively. Assuming that these four matrices are added in the order indicated, we have

$$\begin{aligned} \hat{C}_{11} &= C_{11} + \Delta C_{11}, \\ \|\Delta C_{11}\| &\leq u(3\|\hat{P}_1\| + 3\|\hat{P}_4\| + 2\|\hat{P}_5\| + \|\hat{P}_7\|) + \|F_1 + F_4 - F_5 + F_7\| + O(u^2) \\ &\leq 26 \cdot 2^{k-1}u\|A\| \|B\| + 4(5 \cdot 2^{k-1} + 3c_{k-1})u\|A\| \|B\| + O(u^2) \\ &= (46 \cdot 2^{k-1} + 12c_{k-1})u\|A\| \|B\| + O(u^2). \end{aligned}$$

Clearly, the same bound holds for the other three $\|\Delta C_{ij}\|$ terms. Thus, overall,

$$\hat{C} = AB + \Delta C, \quad \|\Delta C\| \leq (46 \cdot 2^{k-1} + 12c_{k-1})u\|A\| \|B\| + O(u^2).$$

A comparison with (23.15) shows that we need to define the c_k by

$$c_k = 12c_{k-1} + 46 \cdot 2^{k-1}, \quad k > r, \quad c_r = 4^r, \quad (23.16)$$

where $c_r = n_0^2$. Solving this recurrence we obtain

$$\begin{aligned} c_k &= 12^{k-r}4^r + 46 \cdot 2^{k-1} \left(\frac{6^{k-r} - 1}{5} \right) \\ &= \left(\frac{n}{n_0} \right)^{\log_2 12} n_0^2 + \frac{46}{5} \frac{n}{2} \left(\left(\frac{n}{n_0} \right)^{\log_2 6} - 1 \right) \\ &\leq \left(\frac{n}{n_0} \right)^{\log_2 12} (n_0^2 + 5n_0) - 5n, \end{aligned}$$

which gives (23.14). $\square$

The forward error bound for Strassen's method is not of the componentwise form (23.10) that holds for conventional multiplication, which we know it cannot be by Miller's result. One unfortunate consequence is that while the scaling $AB \rightarrow (AD)(D^{-1}B)$, where D is diagonal, leaves (23.10) unchanged, it can alter (23.14) by an arbitrary amount.

The reason for the scale dependence is that Strassen's method adds together elements of A matrix-wide (and similarly for B); for example, in (23.4) A_{11} is added to A_{22} , A_{12} , and A_{21} . This intermingling of elements is particularly undesirable when A or B has elements of widely differing magnitudes because it allows large

errors to contaminate small components of the product. This phenomenon is well illustrated by the example

$$C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & \epsilon \\ \epsilon & \epsilon^2 \end{bmatrix},$$

which is evaluated exactly in floating point arithmetic if we use conventional multiplication. However, Strassen's method computes

$$c_{22} = 2(1 + \epsilon^2) + (\epsilon - \epsilon^2) - 1 - (1 + \epsilon).$$

Because c_{22} involves subterms of order unity, the error $c_{22} - \hat{c}_{22}$ will be of order u . Thus the relative error $|c_{22} - \hat{c}_{22}|/|c_{22}| = O(u/\epsilon^2)$, which is much larger than u if ϵ is small. This is an example where Strassen's method does not satisfy the bound (23.10). For another example, consider the product $X = P_{32}E$, where P_n is the $n \times n$ Pascal matrix (see §28.4) and $e_{ij} \equiv 1/3$. With just one level of recursion in Strassen's method we find in MATLAB that $\max_{ij} |x_{ij} - \hat{x}_{ij}|/|x_{ij}|$ is of order 10^{-5} , so that, again, some elements of the computed product have high relative error.

It is instructive to compare the bound (23.14) for Strassen's method with the weakened, normwise version of (23.10) for conventional multiplication:

$$\|C - \hat{C}\| \leq n^2 u \|A\| \|B\| + O(u^2). \quad (23.17)$$

The bounds (23.14) and (23.17) differ only in the constant term. For Strassen's method, the greater the depth of recursion the bigger the constant in (23.14): if we use just one level of recursion ($n_0 = n/2$) then the constant is $3n^2 + 25n$, whereas with full recursion ($n_0 = 1$) the constant is $6n^{\log_2 12} - 5n \approx 6n^{3.585} - 5n$. It is also interesting to note that the bound for Strassen's method (minimal for $n_0 = n$) is not correlated with the operation count (minimal for $n_0 = 8$).

Our conclusion is that Strassen's method has less favourable stability properties than conventional multiplication in two respects: it satisfies a weaker error bound (normwise rather than componentwise) and it has a larger constant in the bound (how much larger depending on n_0).

Another interesting property of Strassen's method is that it always involves some genuine subtractions (assuming that all additions are of nonzero terms). This is easily deduced from the formulae (23.4). This makes Strassen's method unattractive in applications where all the elements of A and B are nonnegative (for example, in Markov processes). Here, conventional multiplication yields low componentwise relative error because, in (23.10), $|A||B| = |AB| = |C|$, yet comparable accuracy cannot be guaranteed for Strassen's method.

An analogue of Theorem 23.2 holds for Winograd's variant of Strassen's method.

Theorem 23.3. *Let $A, B \in \mathbb{R}^{n \times n}$, where $n = 2^k$. Suppose that $C = AB$ is computed by Winograd's variant (23.6) of Strassen's method and that $n_0 = 2^r$ is the threshold at which conventional multiplication is used. The computed product $\hat{C}$ satisfies*

$$\|C - \hat{C}\| \leq \left[\left(\frac{n}{n_0} \right)^{\log_2 18} (n_0^2 + 6n_0) - 6n \right] u \|A\| \|B\| + O(u^2). \quad (23.18)$$

Proof. The proof is analogous to that of Theorem 23.2, but more tedious. It suffices to analyse the computation of C_{12} , and the recurrence corresponding to (23.16) is

$$c_k = 18c_{k-1} + 89 \cdot 2^{k-1}, \quad k > r, \quad c_r = 4^r. \quad \square$$

Note that the bound for the Winograd–Strassen method has exponent $\log_2 18 \approx 4.170$ in place of $\log_2 12 \approx 3.585$ for Strassen’s method, suggesting that the price to be paid for a reduction in the number of additions is an increased rate of error growth. All the comments above about Strassen’s method apply to the Winograd variant.

Two further questions are suggested by the error analysis:

- How do the actual errors compare with the bounds?
- Which formulae are the more accurate in practice, Strassen’s or Winograd’s variant?

To give some insight we quote results obtained with a single precision Fortran 90 implementation ($u \approx 6 \times 10^{-8}$) of the two methods (the code is easy to write if we exploit the language’s dynamic arrays and recursive procedures). We take random $n \times n$ matrices A and B and look at $\|AB - fl(AB)\|/(\|A\| \|B\|)$ for $n_0 = 1, 2, \dots, 2^k = n$ (note that this is not the relative error, since the denominator is $\|A\| \|B\|$ instead of $\|AB\|$, and note that $n_0 = n$ corresponds to conventional multiplication). Figure 23.2 plots the results for one random matrix of order 1024 from the uniform $[0, 1]$ distribution and another matrix of the same size from the uniform $[-1, 1]$ distribution. The error bound (23.14) for Strassen’s method is also plotted. Two observations can be made.

- Winograd’s variant can be more accurate than Strassen’s formulae, for all n_0 , despite its larger error bound.
- The error bound overestimates the actual error by a factor up to 1.8×10^6 for $n = 1024$, but the variation of the errors with n_0 is roughly as predicted by the bound.

23.2.3. Bilinear Noncommutative Algorithms

Bini and Lotti [110, 1980] have analysed the stability of bilinear noncommutative algorithms in general. They prove the following result.

Theorem 23.4 (Bini and Lotti). *Let $A, B \in \mathbb{R}^{n \times n}$ ($n = h^k$) and let the product $C = AB$ be formed by a recursive application of the bilinear noncommutative algorithm (23.7), which multiplies $h \times h$ matrices using t nonscalar multiplications. The computed product $\hat{C}$ satisfies*

$$\|C - \hat{C}\| \leq \alpha u n^{\log_h \beta} \log_h n \|A\| \|B\| + O(u^2), \quad (23.19)$$

where α and β are constants that depend on the number of nonzero terms in the matrices U , V , and W that define the algorithm. $\square$

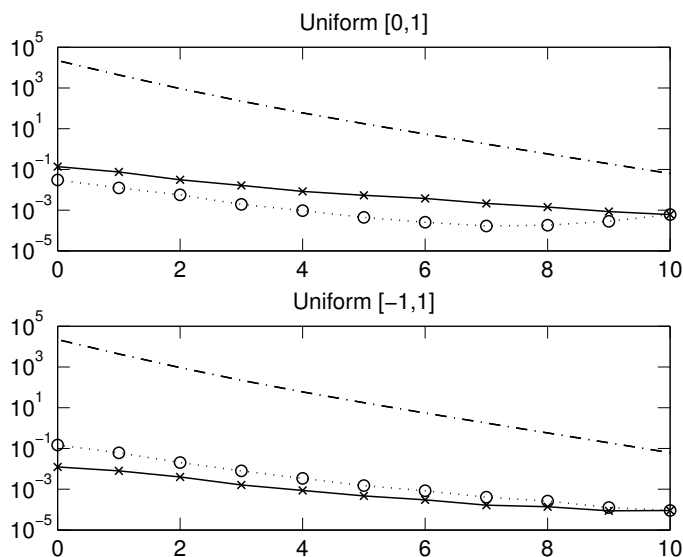


Figure 23.2. Errors for Strassen's method with two random matrices of dimension $n = 1024$. Strassen's formulae: "x", Winograd's variant: "o". X-axis contains $\log_2$ of recursion threshold n_0 , $1 \leq n_0 \leq n$. Dot-dash line is error bound for Strassen's formulae.

The precise definition of α and β is given in [110, 1980]. If we take $k = 1$, so that $h = n$, and if the basic algorithm (23.7) is chosen to be conventional multiplication, then it turns out that $\alpha = n - 1$ and $\beta = n$, so the bound of the theorem becomes $(n - 1)nu\|A\| \|B\| + O(u^2)$, which is essentially the same as (23.17). For Strassen's method, $h = 2$ and $t = 7$, and $\alpha = 5$, $\beta = 12$, so the theorem produces the bound $5un^{\log_2 12} \log_2 n \|A\| \|B\| + O(u^2)$, which is a factor $\log_2 n$ larger than (23.14) (with $n_0 = 1$). This extra weakness of the bound is not surprising given the generality of Theorem 23.4.

Bini and Lotti consider the set of all bilinear noncommutative algorithms that form 2×2 products in 7 multiplications and that employ integer constants of the form $\pm 2^i$, where i is an integer (this set breaks into 26 equivalence classes). They show that Strassen's method has the minimum exponent β in its error bound in this class (namely, $\beta = 12$). In particular, Winograd's variant of Strassen's method has $\beta = 18$, so Bini and Lotti's bound has the same exponent $\log_2 18$ as in Theorem 23.3.

23.2.4. The 3M Method

A simple example reveals a fundamental weakness of the 3M method. Consider the computation of the scalar

$$z = x + iy = (\theta + i/\theta)^2 = \theta^2 - 1/\theta^2 + 2i.$$

In floating point arithmetic, if y is computed in the usual way, as $y = \theta(1/\theta) + (1/\theta)\theta$, then no cancellation occurs and the computed $\hat{y}$ has high relative accuracy:

$|\hat{y} - y|/|y| = O(u)$. The 3M method computes

$$y = \left(\theta + \frac{1}{\theta}\right)\left(\theta + \frac{1}{\theta}\right) - \theta^2 - \frac{1}{\theta^2}.$$

If $|\theta|$ is large this formula expresses a number of order 1 as the difference of large numbers. The computed $\hat{y}$ will almost certainly be contaminated by rounding errors of order $u\theta^2$, in which case the relative error is large: $|\hat{y} - y|/|y| = O(u\theta^2)$. However, if we measure the error in $\hat{y}$ relative to z , then it is acceptably small: $|\hat{y} - y|/|z| = O(u)$. This example suggests that the 3M method may be stable, but in a weaker sense than for conventional multiplication.

To analyse the general case, consider the product $C_1 + iC_2 = (A_1 + iA_2)(B_1 + iB_2)$, where $A_k, B_k, C_k \in \mathbb{R}^{n \times n}$, $k = 1:2$. Using (23.10) we find that the computed product from conventional multiplication,

$$\hat{C} = fl(A_1B_1 - A_2B_2 + i(A_1B_2 + A_2B_1)),$$

satisfies

$$|C_1 - \hat{C}_1| \leq (n+1)u(|A_1||B_1| + |A_2||B_2|) + O(u^2), \quad (23.20)$$

$$|C_2 - \hat{C}_2| \leq (n+1)u(|A_1||B_2| + |A_2||B_1|) + O(u^2). \quad (23.21)$$

For the 3M method C_1 is computed in the conventional way, and so (23.20) holds. It is straightforward to show that $\hat{C}_2$ satisfies

$$|C_2 - \hat{C}_2| \leq (n+4)u[(|A_1| + |A_2|)(|B_1| + |B_2|) + |A_1||B_1| + |A_2||B_2|] + O(u^2). \quad (23.22)$$

Two notable features of the bound (23.22) are as follows. First, it is of a different and weaker form than (23.21); in fact, it exceeds the sum of the bounds (23.20) and (23.21). Second and more pleasing, it retains the property of (23.20) and (23.21) of being invariant under diagonal scalings

$$C = AB \rightarrow D_1AD_2 \cdot D_2^{-1}BD_3 = D_1CD_3, \quad D_j \text{ diagonal},$$

in the sense that the upper bound ΔC_2 in (23.22) scales also according to $D_1\Delta C_2D_3$. (The “hidden” second-order terms in (23.20)–(23.22) are invariant under these diagonal scalings.)

The disparity between (23.21) and (23.22) is, in part, a consequence of the differing numerical cancellation properties of the two methods. It is easy to show that there are always subtractions of like-signed numbers in the 3M method, whereas if A_1, A_2, B_1 , and B_2 have nonnegative elements (for example) then no numerical cancellation takes place in conventional multiplication.

We can define a measure of stability with respect to which the 3M method matches conventional multiplication by taking norms in (23.21) and (23.22). We obtain the weaker bounds

$$\|C_2 - \hat{C}_2\|_\infty \leq 2(n+1)u\|A\|_\infty\|B\|_\infty + O(u^2), \quad (23.23)$$

$$\|C_2 - \hat{C}_2\|_\infty \leq 4(n+4)u\|A\|_\infty\|B\|_\infty + O(u^2) \quad (23.24)$$

(having used $\| |A_1| + |A_2| \|_\infty \leq \sqrt{2} \|A_1 + iA_2\|_\infty$). Combining these with an analogous weakening of (23.20), we find that for both conventional multiplication and the 3M method the computed complex matrix $\hat{C}$ satisfies

$$\|C - \hat{C}\|_\infty \leq c_n u \|A\|_\infty \|B\|_\infty + O(u^2),$$

where $c_n = O(n)$.

The conclusion is that the 3M method produces a computed product $\hat{C}$ whose imaginary part may be contaminated by relative errors much larger than those for conventional multiplication (or, equivalently, much larger than can be accounted for by small componentwise perturbations in the data A and B). However, if the errors are measured relative to $\|A\| \|B\|$, which is a natural quantity to use for comparison when employing matrix norms, then they are just as small as for conventional multiplication.

It is straightforward to show that if the 3M method is implemented using Strassen's method to form the real matrix products, then the computed complex product $\hat{C}$ satisfies the same bound (23.14) as for Strassen's method itself, but with an extra constant multiplier of 6 and with 4 added to the expression inside the square brackets.

23.3. Notes and References

A good introduction to the construction of fast matrix multiplication methods is provided by the papers of Pan [916, 1984] and Laderman, Pan, and Sha [761, 1992].

Harter [549, 1972] shows that Winograd's formula (23.2) is the best of its kind, in a certain precise sense.

How does one derive formulae such as those of Winograd and Strassen, or that in the 3M method? Inspiration and ingenuity seem to be the key. A fairly straightforward, but still not obvious, derivation of Strassen's method is given by Yuval [1276, 1978]. Gustafson and Aluru [534, 1996] develop algorithms that systematically search for fast algorithms, taking advantage of a parallel computer. In an exhaustive search taking 21 hours of computation time on a 256 processor nCUBE 2, they were able to find 12 methods for multiplying 2 complex numbers in 3 multiplications and 5 additions; they could not find a method with fewer additions, thus proving that such a method does not exist. However, they estimate that a search for Strassen's method on a 1024 processor nCUBE 2 would take many centuries, even using aggressive pruning rules, so human ingenuity is not yet redundant!

To obtain a useful implementation of Strassen's method a number of issues have to be addressed, including how to program the recursion, how best to handle rectangular matrices of arbitrary dimension (since the basic method is defined only for square matrices of dimension a power of 2), and how to cater for the extra storage required by the method. These issues are discussed by Bailey [52, 1988], Bailey, Lee, and Simon [56, 1991], Fischer [414, 1974], Higham [592, 1990], Kreczmar [750, 1976], and Spieß [1058, 1976], among others. Douglas, Heroux, Shishman, and Smith [351, 1994] present a portable Fortran implementation of

Winograd's variant of Strassen's method for real and complex matrices, with a level-3 BLAS interface; they take care to use a minimal amount of extra storage (about $2n^3/3$ elements of extra storage is required when multiplying $n \times n$ matrices).

Higham [592, 1990] shows how Strassen's method can be used to produce algorithms for all the level-3 BLAS operations that are asymptotically faster than the conventional algorithms. Most of the standard algorithms in numerical linear algebra remain stable (in an appropriately weakened sense) when fast level-3 BLAS are used. See, for example, Chapter 13, §19.5, and Problems 12.4 and 14.2.

Knight [740, 1995] shows how to choose the recursion threshold to minimize the operation count of Strassen's method for rectangular matrices. He also shows how to use Strassen's method to compute the QR factorization of an $m \times n$ matrix in $O(mn^{1.838})$ operations instead of the usual $O(mn^2)$.

Bailey, Lee, and Simon [56, 1991] substituted their Strassen's method code for a conventionally coded BLAS3 subroutine **SGEMM** and tested LAPACK's LU factorization subroutine **SGETRF** on a Cray Y-MP. They obtained speed improvements for matrix dimensions 1024 and larger.

Kaporin [709, 1999] investigates a fast matrix multiplication formula of Pan from 1972 and shows that it can be implemented so as to be competitive with Strassen's method in terms of both practical efficiency and numerical accuracy. This method deserves further investigation.

The Fortran 95 standard includes an intrinsic function **MATMUL** that returns the product of its two matrix arguments. The standard does not specify which method is to be used for the multiplication. An IBM compiler supports the use of Winograd's variant of Strassen's method, via an optional third argument to **MATMUL** (an extension to Fortran 95) [352, 1994].

Not only multiplication but also division of complex numbers can be done with fewer real multiplications and divisions than by the obvious formulae. The division $(a+ib)/(c+id)$ can be done with six real multiplications and divisions (see (27.1)) and $1/(c+id)$ and $a/(c+id)$ can both be done in four real multiplications and divisions. These operation counts, and that for (23.8), are optimal, as shown by Alt and van Leeuwen [15, 1981] and Lickteig [787, 1987].

Brent was the first to point out the possible instability of Winograd's method [159, 1970]. He presented a full error analysis (including Theorem 23.1) and showed that scaling ensures stability.

An error analysis of Strassen's method was given by Brent in 1970 in an unpublished technical report that has not been widely cited [158, 1970]. Section 23.2.2 is based on Higham [592, 1990].

When AB is to be formed by Strassen's method and the elements of A and B vary widely in magnitude Dumitrescu [367, 1998] suggests computing $D_1(D_1^{-1}A \cdot BD_2^{-1})D_2$, where the diagonal matrices D_1 and D_2 are chosen to equilibrate the rows of A and the columns of B ; he shows that this scaling can improve the accuracy of the result.

According to Knuth, the 3M formula was suggested by P. Ungar in 1963 [744, 1998, p. 706]. It is analogous to a formula of Karatsuba and Ofman [711, 1963] for squaring a $2n$ -digit number using three squarings of n -digit numbers. That three multiplications (or divisions) are *necessary* for evaluating the product of two

complex numbers was proved by Winograd [1250, 1971].

Section 23.2.4 is based on Higham [599, 1992].

The answer to the question, “What method should we use to multiply complex matrices?” depends on the desired accuracy and speed. In a Fortran environment an important factor affecting the speed is the relative efficiency of real and complex arithmetic, which depends on the compiler and the computer (complex arithmetic is automatically converted by the compiler into real arithmetic). For a discussion and some statistics see [599, 1992].

The efficiency of Winograd’s method is very machine dependent. Bjørstad, Manne, Sørøvik, and Vajteršić [134, 1992] found the method useful on the MasPar MP-1 parallel machine, on which floating point multiplication takes about three times as long as floating point addition at 64-bit precision. They also implemented Strassen’s method on the MP-1 (using Winograd’s method at the bottom level of recursion) and obtained significant speedups over conventional multiplication for dimensions as small as 256.

As noted in §23.1, Strassen [1093, 1969] gave not only a method for multiplying $n \times n$ matrices in $O(n^{\log_2 7})$ operations, but also a method for inverting an $n \times n$ matrix with the same asymptotic cost. The method is described in Problem 23.8. For more on Strassen’s inversion method see §26.3.2, Bailey and Ferguson [50, 1988], and Balle, Hansen, and Higham [60, 1993].

Problems

23.1. (Knight [740, 1995]) Suppose we have a method for multiplying $n \times n$ matrices in $O(n^\alpha)$ operations, where $2 < \alpha < 3$. Show that if A is $m \times n$ and B is $n \times p$ then the product AB can be formed in $O(n_1^{\alpha-2} n_2 n_3)$ operations, where $n_1 = \min(m, n, p)$ and n_2 and n_3 are the other two dimensions.

23.2. Work out the operation count for Winograd’s method applied to $n \times n$ matrices.

23.3. Let $S_n(n_0)$ denote the operation count (additions plus multiplications) for Strassen’s method applied to $n \times n$ matrices, with recursion down to the level of $n_0 \times n_0$ matrices. Assume that n and n_0 are powers of 2. For large n , estimate $S_n(8)/S_n(n)$ and $S_n(1)/S_n(8)$ and explain the significance of these ratios (use (23.5)).

23.4. (Knight [740, 1995]) Suppose that Strassen’s method is used to multiply an $m \times n$ matrix by an $n \times p$ matrix, where $m = a2^j$, $n = b2^j$, $p = c2^j$, and that conventional multiplication is used once any dimension is 2^r or less. Show that the operation count is $\alpha 7^j + \beta 4^j$, where

$$\alpha = 2abc \left(\frac{8}{7}\right)^r + \frac{5}{3}(ab + bc + ac) \left(\frac{4}{7}\right)^r,$$

$$\beta = -\frac{1}{3}(5ab + 5bc + 8ac).$$

Show that by setting $r = 0$ and $a = 1$ a special case of the result of Problem 23.1 is obtained.

23.5. Compare and contrast Winograd's inner product formula for $n = 2$ with the imaginary part of the 3M formula (23.8).

23.6. Prove the error bound described at the end of §23.2.4 for the combination of the 3M method and Strassen's method.

23.7. Two fast ways to multiply complex matrices are (a) to apply the 3M method to the original matrices and to use Strassen's method to form the three real matrix products, and (b) to use Strassen's method with the 3M method applied at the bottom level of recursion. Investigate the merits of the two approaches with respect to speed and storage.

23.8. Strassen [1093, 1969] gives a method for inverting an $n \times n$ matrix in $O(n^{\log_2 7})$ operations. Assume that n is even and write

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \in \mathbb{R}^{n \times n}, \quad A_{ij} \in \mathbb{R}^{m \times m}, \quad n = 2m.$$

The inversion method is based on the following formulae:

$$\begin{aligned} P_1 &= A_{11}^{-1}, & P_2 &= A_{21}P_1, \\ P_3 &= P_1A_{12}, & P_4 &= A_{21}P_3, \\ P_5 &= P_4 - A_{22}, & P_6 &= P_5^{-1}, \\ A^{-1} &= \begin{bmatrix} P_1 - P_3P_6P_2 & P_3P_6 \\ P_6P_2 & -P_6 \end{bmatrix}. \end{aligned}$$

The matrix multiplications are done by Strassen's method and the inversions determining P_1 and P_6 are done by recursive invocations of the method itself. (a) Verify these formulae, using a block LU factorization of A , and show that they permit the claimed $O(n^{\log_2 7})$ complexity. (b) Show that if A is upper triangular, Strassen's method is equivalent to (the unstable) Method 2B of §14.2.2.

(For a numerical investigation into the stability of Strassen's inversion method, see §26.3.2.)

23.9. Find the inverse of the block upper triangular matrix

$$\begin{bmatrix} I & A & 0 \\ 0 & I & B \\ 0 & 0 & I \end{bmatrix}.$$

Deduce that matrix multiplication can be reduced to matrix inversion.

23.10. (RESEARCH PROBLEM) Carry out extensive numerical experiments to test the accuracy of Strassen's method and Winograd's variant (cf. the results at the end of §23.2.2).